

Section 8.8: Feature Flags & Progressive Delivery

Module 8 of 8 · Flag Taxonomy, Lifecycle Management & Safe Rollbacks

Executive Overview

Feature flags decouple deployment from release. Code is shipped dark (deployed but disabled), then enabled progressively — first to internal users, then a canary percentage, then fully. This enables instant rollback without a code deployment, A/B experiments with statistical rigour, and kill switches for emergency response. Getting the lifecycle wrong, however, creates flag debt that degrades code quality and test reliability over time.

Part 1: Feature Flag Taxonomy

1.1 Four Flag Types — When to Use Each

Type	Purpose	Lifetime	Example
Release Toggle	Ship code dark; enable when ready	Days to weeks	New checkout flow
Experiment Toggle	A/B or multivariate testing	Days to weeks	Pricing variant test
Ops Toggle (Kill Switch)	Emergency disable; rate control	Indefinite	Payment processing kill switch
Permission Toggle	Control access by user segment	Indefinite (while feature exists)	Beta feature for enterprise users

Anti-pattern: Using a Release Toggle as a permanent Permission Toggle. Release toggles should be removed after full rollout. Permission toggles are configured permanently and managed as product features.

1.2 Release Toggles

```
# Feature flag evaluation – abstracted behind interface
from feature_flags import FlagClient

flags = FlagClient()

def checkout(cart: Cart, user: User) -> CheckoutResult:
    if flags.is_enabled('new_checkout_flow', context={'user_id': user.id}):
        return new_checkout_service.process(cart, user)
    else:
        return legacy_checkout_service.process(cart, user)
```

Rollout configuration (LaunchDarkly / OpenFeature format):

```
flag: new_checkout_flow
type: release
targeting:
  - rule: user.id in internal_users_list → serve: true      # Step 1: Internal
  - rule: user.id % 100 < 5 → serve: true                  # Step 2: 5% canary
  - rule: user.id % 100 < 50 → serve: true                  # Step 3: 50%
  - default: false                                          # Everyone else: old
flow
```

1.3 Experiment Toggles

```
# Multi-variant experiment – returns variant string, not bool
def render_pricing_page(user: User) -> PricingPageResponse:
    variant = flags.get_variant('pricing_experiment', user_id=user.id)
    # variant ∈ {'control', 'discount_5', 'discount_10', 'annual_promo'}

    metrics.track('pricing_page_view', user_id=user.id, variant=variant)

    match variant:
        case 'control':      return PricingPage(discounts=[])
        case 'discount_5':   return PricingPage(discounts=[Discount(5)])
        case 'discount_10':  return PricingPage(discounts=[Discount(10)])
        case 'annual_promo': return PricingPage(show_annual_badge=True)
        case _:              return PricingPage(discounts=[]) # Safe default
```

Statistical significance requirement:

```
Experiment config:
- Control:      25% of users
- Variant A:    25% of users
- Variant B:    25% of users
- Variant C:    25% of users
```

Minimum sample size for significance ($p < 0.05$, power = 0.8):
Baseline conversion: 3%
Minimum detectable effect: 0.5% (relative 17% lift)
Required sample per variant: ~15,000 users
At 10K daily active users: ~6 days per experiment

1.4 Ops Toggles / Kill Switches

```
class PaymentController:
    def process_payment(self, req: PaymentRequest) -> PaymentResponse:
        # Hard kill switch – checked before any work
        if not flags.is_enabled('payment_processing'):
            return PaymentResponse(
                status='SERVICE_UNAVAILABLE',
                message='Payment processing temporarily unavailable. Please retry
in a few minutes.',
                retry_after=300
            )

        # Soft kill switch – degrade gracefully
        if not flags.is_enabled('3ds_authentication'):
            # Continue without 3DS if that service is degraded
            return self._process_without_3ds(req)

        return self._process_with_3ds(req)
```

1.5 Permission Toggles

```
def get_user_features(user: User) -> FeatureSet:
    context = {
        'user_id': user.id,
        'plan': user.plan,
        'account_age_days': user.account_age_days(),
        'beta_tester': user.id in beta_tester_list
    }

    return FeatureSet(
        advanced_analytics = flags.is_enabled('advanced_analytics', context),
        api_access          = flags.is_enabled('api_access', context),
        bulk_export         = flags.is_enabled('bulk_export', context),
        white_label         = flags.is_enabled('white_label', context)
    )
```

Part 2: Flag Lifecycle Management

2.1 Complete Lifecycle

- | | |
|-------------|---|
| 1. DEFINED | Flag declared in system; code written behind it; tests cover both paths |
| 2. DARK | Deployed to production; all users see old behavior (flag off) |
| 3. INTERNAL | Enabled for internal test users / beta list (0.1%) |
| 4. CANARY | 1-5% of real users; monitor metrics carefully |
| 5. RAMP | 10% → 25% → 50% → 75%; automated health gates |
| 6. FULL | 100% of users; old code path still exists |
| 7. CLEANUP | Remove flag and old code path; delete configuration |

```
# Automated rollout progression with health gates
class FeatureFlagRollout:
    RAMP_SCHEDULE = [0.01, 0.05, 0.10, 0.25, 0.50, 0.75, 1.0]
    GATE_DURATION_MINUTES = 30
    ERROR_RATE_THRESHOLD = 0.02    # 2% error rate → halt

    def auto_rollout(self, flag_name: str):
        for percentage in self.RAMP_SCHEDULE:
            self.set_flag_percentage(flag_name, percentage)
            print(f"[{flag_name}] Enabled for {percentage*100:.0f}% of users")

            time.sleep(self.GATE_DURATION_MINUTES * 60)

            # Health gate
            metrics = self.collect_metrics(flag_name,
window_minutes=self.GATE_DURATION_MINUTES)

            if metrics.error_rate > self.ERROR_RATE_THRESHOLD:
                self.rollback(flag_name)
                self.alert(f"[{flag_name}] Auto-rollback triggered:
error_rate={metrics.error_rate:.2%}")
                return

            if metrics.p99_latency_ms > metrics.baseline_p99_ms * 1.2:    # 20%
regression
                self.rollback(flag_name)
                self.alert(f"[{flag_name}] Auto-rollback triggered: latency
regression")
                return

            print(f"[{flag_name}] Full rollout complete ✓")
            self.schedule_cleanup_ticket(flag_name, days=7)
```

2.2 Technical Debt Prevention

Flag debt accumulation:

- 100 flags × 2 code paths each = 200 code paths to reason about
- Test matrix: $2^{10} = 1,024$ combinations for 10 simultaneous flags
- Performance: Each flag check adds 1-5ms if hitting network (cache helps)
- Confusion: "Is this flag safe to clean up? Which team owns it?"

Prevention: Treat flags as inventory items with expiry dates

```
# Flag registry with mandatory metadata
@dataclass
class FeatureFlag:
    name: str
    flag_type: Literal['release', 'experiment', 'ops', 'permission']
    owner: str          # Team or individual responsible
    jira_ticket: str     # Cleanup ticket number
    expires_at: date | None # None = permanent (ops/permission only)
    description: str

# CI check: block PRs that reference flags older than 30 days (release type)
def check_flag_debt(flag_name: str, code_file: str):
    flag = flag_registry.get(flag_name)
    if flag.flag_type == 'release' and flag.age_days() > 30:
        raise CIFailure(
            f"Flag '{flag_name}' in {code_file} is {flag.age_days()} days old. "
            f"Create a cleanup PR or extend the flag's planned expiry. "
            f"Owner: {flag.owner}, Ticket: {flag.jira_ticket}"
        )
```

Part 3: Rollout Patterns

3.1 Canary Deployment via Feature Flags

Traditional canary: route 5% of traffic to new pod version
→ Requires infrastructure changes; slow; binary rollback

Flag-based canary: deploy new code to all pods; enable via flag for 5%
→ Instant enable/disable; no infrastructure changes; gradual ramp
→ Can target specific user segments (e.g., test users in US only)

```
# Targeting rules: US users, non-mobile, account > 30 days old
canary_rules = FlagRules([
    Rule(condition="user.country == 'US'", serve=True),
    Rule(condition="user.device_type == 'desktop'", serve=True),
```

```

    Rule(condition="user.account_age_days >= 30", serve=True),
    DefaultRule(serve=False)
  })
# Effect: flag only enables for users matching ALL rules, up to rollout %

```

3.2 Comparison: Deployment Strategies

Strategy	Mechanism	Rollback Speed	Traffic Scope	Best For
Big Bang	Deploy all at once	Slow (re-deploy)	100% immediately	Internal tools, trivial changes
Rolling	Replace pods gradually	Medium (re-deploy)	Incremental pods	Stateless services; no dark launch
Blue/Green	Two environments; flip DNS	Fast (DNS TTL)	All at once	DB migrations; needs 2x infra
Canary	Small % to new version	Fast (route 0% to canary)	Small % → 100%	Risk reduction; needs routing logic
Flag-based Canary	New code everywhere; flag enables	Instant (flip flag)	User-targeted %	Best flexibility; recommended default

Part 4: Interview Design Problems

Problem 1 — Caching Layer Rollback

Question: Your team deployed a feature flag for a new Redis-based caching layer. After 2 weeks at 100% rollout, cache hit rate is 40% lower than expected and p99 latency is 3x worse. How do you safely roll back while preserving data integrity?

Solution:

```

# Step 1: Immediate diagnosis – is the problem the cache or something else?
def diagnose(self) -> DiagnosisResult:
    metrics_before_cache = self.get_metrics('cache_miss_fallthrough_latency')
    metrics_cache_only    = self.get_metrics('cache_hit_latency')

    # If cache_miss_latency is normal but p99 is high → cache hits are slow

```

```

# If cache_hit_latency is normal but p99 is high → high miss rate issue
return DiagnosisResult(...)

# Step 2: Gradual disable (NOT instant kill) to avoid sudden DB load spike
class SafeCacheRollback:
    def execute(self):
        # Phase 1: Route 10% of traffic to old path (verify DB can handle load)
        self.set_flag_percentage('new_cache_layer', 90)
        time.sleep(120)
        self._check_db_connections_healthy()

        # Phase 2: 50% old path
        self.set_flag_percentage('new_cache_layer', 50)
        time.sleep(120)
        self._check_db_connections_healthy()

        # Phase 3: 100% old path
        self.set_flag_percentage('new_cache_layer', 0)

        # Phase 4: Warm old cache before traffic fully switches
        self._warm_legacy_cache()

    def _warm_legacy_cache(self):
        """Prime the legacy cache with hot keys from DB to prevent cache
stampedede."""
        hot_keys = self.analytics.get_top_keys(limit=10_000, time_window_hours=1)
        batch_loader = BatchCacheLoader(target=self.legacy_cache)
        batch_loader.load(hot_keys, source=self.database)

    def _check_db_connections_healthy(self):
        pool_utilisation = self.db_monitor.connection_pool_utilisation()
        if pool_utilisation > 0.8:
            raise RollbackHaltError(
                f"DB connection pool at {pool_utilisation:.0%} – pause rollback"
            )

```

Data integrity during dual-write period:

```

class DualCacheWriter:
    """Used during transition: writes to both old and new cache."""

    def set(self, key: str, value: Any, ttl_seconds: int):
        # Write to new cache (async – don't wait for confirmation during rollback)
        try:
            self.new_cache.set(key, value, ttl=ttl_seconds)
        except Exception:
            pass # New cache is being retired; errors are expected

        # Write to old cache (sync – must succeed)
        self.old_cache.set(key, value, ttl=ttl_seconds)

    def get(self, key: str) -> Any:
        # During rollback: read from old cache (authoritative)

```

```

value = self.old_cache.get(key)
if value is not None:
    return value
# Miss: fall through to DB, prime old cache
value = self.database.get(key)
if value is not None:
    self.old_cache.set(key, value)
return value

```

Problem 2 — A/B Test for New Recommendation Algorithm

Question: You want to test a new ML recommendation algorithm. 20% of users get the new algorithm; 80% get the old one. How do you ensure statistical validity and prevent user experience inconsistency (seeing different recommendations on each page load)?

Solution:

```

# Sticky assignment: user always gets same variant
class ExperimentAssignment:
    def get_variant(self, user_id: str, experiment: str) -> str:
        # Deterministic: same user_id + experiment -> same bucket
        bucket = murmur_hash(f"{user_id}:{experiment}") % 100

        if bucket < 20:    return 'new_algorithm'    # 20%
        else:              return 'control'          # 80%

# This is sticky by construction: hash(user_id) never changes
# Same user sees same variant on every page load, every device

# Usage
def get_recommendations(user_id: str, page_context: PageContext) -> list[Item]:
    variant = assignment.get_variant(user_id, 'recommendation_algo_v2')

    if variant == 'new_algorithm':
        items = ml_recommender_v2.recommend(user_id, page_context)
    else:
        items = ml_recommender_v1.recommend(user_id, page_context)

    # Track exposure for analysis
    experiment_tracker.track_exposure(
        experiment='recommendation_algo_v2',
        user_id=user_id,
        variant=variant
    )

    return items

```

Statistical guard rails:


```

class ExperimentGuardRails:
    def check_novelty_bias(self, experiment: str, window_hours: int = 24) -> bool:
        """First-day effect: new UI is exciting, inflating metrics. Check it."""
        day_1_ctr = metrics.get_ctr(experiment, 'new_algorithm',
days_since_start=[0])
        day_7_ctr = metrics.get_ctr(experiment, 'new_algorithm',
days_since_start=[6])

        if day_1_ctr / day_7_ctr > 1.3: # > 30% novelty deflation
            self.alert("Novelty bias detected – extend experiment duration")
            return False
        return True

    def check_sample_ratio_mismatch(self, experiment: str) -> bool:
        """Ensure traffic split is actually 20/80, not skewed by bugs."""
        actual_split = metrics.get_variant_distribution(experiment)
        expected = {'new_algorithm': 0.20, 'control': 0.80}

        for variant, expected_pct in expected.items():
            actual_pct = actual_split[variant]
            if abs(actual_pct - expected_pct) > 0.02: # > 2% deviation
                self.alert(f"SRM detected: expected {expected_pct:.0%}, got
{actual_pct:.0%}")
            return False
        return True

```

Problem 3 — Feature Flag System Design

Question: Design the internal architecture of a feature flag evaluation service. It must handle 500K flag evaluations per second with P99 < 5ms. How do you architect it?

Solution:

Architecture: SDK → Local Cache → Flag Service → Config Store

SDK (in every service, language-native):

- Evaluates flags in-memory: 0ms latency
- Background polling: refresh flag config every 30s
- Streaming (SSE/WebSocket): receive push updates on flag changes
- Fallback: if service unreachable, use last-known config

Flag Service (centralised):

- Stores flag configs in Postgres (source of truth)
- Redis cache layer for hot flags (< 1ms reads)
- Fan-out: publish flag changes to all SDK instances via SSE/WebSocket
- Rate: 1 change/hour typical; streaming has no throughput concern

Why this achieves 500K evals/sec at < 5ms:

- Flag evaluations happen in-process (SDK), not via network call

- No network hop = 0ms overhead (plus evaluation logic < 1ms)
- Config refresh is async; never blocks an evaluation

```
# SDK evaluation (in-process, < 0.1ms)
class FeatureFlagSDK:
    def __init__(self, service_url: str, sdk_key: str):
        self._flags: dict[str, FlagConfig] = {}
        self._lock = threading.RLock()
        self._start_background_sync(service_url, sdk_key)

    def is_enabled(self, flag_name: str, context: dict) -> bool:
        with self._lock:
            config = self._flags.get(flag_name)

            if config is None:
                return False # Unknown flag = disabled (safe default)

            # Evaluate targeting rules in-process
            for rule in config.rules:
                if rule.matches(context):
                    return rule.serve_value

            return config.default_value

    def _start_background_sync(self, url: str, key: str):
        # Background thread: SSE stream for instant push updates
        def listen():
            for event in sse_client.connect(f"{url}/stream",
                headers={"Authorization": key}):
                if event.type == 'flag_updated':
                    updated = FlagConfig.from_json(event.data)
                    with self._lock:
                        self._flags[updated.name] = updated

        threading.Thread(target=listen, daemon=True).start()
        # Also poll as backup in case SSE stream drops
        self._start_polling_fallback(url, key)
```

Quick Reference

Flag Type	Lifetime	Cleanup Needed?	Use When
Release	Days–weeks	Yes — after full rollout	New feature going live
Experiment	Days–weeks	Yes — after conclusion	A/B test
Ops / Kill Switch	Indefinite	No	Emergency disable

Flag Type	Lifetime	Cleanup Needed?	Use When
Permission	Indefinite	No (product feature)	Plan-based access control

Strategy	Rollback Speed	Targeting	Use When
Flag-based canary	Instant (flip flag)	User-level	Default for user-facing features
Blue/Green	DNS TTL (~60s)	All-or-nothing	DB schema migrations
Rolling	Minutes (re-deploy)	Pod-level	Infrastructure changes

Metric	Threshold to Halt Rollout
Error rate increase	> 1–2% above baseline
P99 latency increase	> 20% above baseline
Conversion rate drop	> 5% relative (statistical significance required)
SRM (sample ratio mismatch)	> 2% deviation from expected split